



FHM
World class pioneers



CLAUDE CERTIFIED ARCHITECT FOUNDATIONS (CCA-F)

Exam guide, study roadmap & tips to pass on your first attempt

KienLT36 & HetTV
Presenters

FPT SOFTWARE | 2026



CCA-F EXAM GUIDE

AGENDA

Four sections — from exam overview to Q&A

01 What is CCA-F?

02 Exam guide & key tips to watch

03 CCA-F Update

04 Tips & experience sharing

05 Q&A

01

What is CCA-F?

The credential, what it validates, and who it's for

THE CREDENTIAL

Anthropic's first official Claude technical certification



A technical exam for Anthropic Partners building production applications with Claude · launched March 12, 2026 · valid for 12 months

WHAT IT VALIDATES

[View details →](#)

01

Claude API & Agent SDK

Build agentic loops and multi-agent systems on the Claude Agent SDK.

✓ Core building blocks

02

Claude Code

Configure CLAUDE.md, Agent Skills, slash commands & CI/CD.

✓ Dev workflows

03

Model Context Protocol

Design MCP servers and tool interfaces with safe boundaries.

✓ Tools & integration

02

Exam guide & key tips

Format, scoring, registration, and the 5 exam domains

Exam Overview



PASSING SCORE

720

out of 1,000 · $\geq 44 / 60$ to pass

60	Questions
120	Minutes
\$125	Exam fee

DOMAIN WEIGHTING

[View details →](#)

27%	20%	20%	18%	15%
Agentic Architecture & Orchestration	Claude Code Configuration & Workflows	Prompt Engineering & Structured Output	Tool Design & MCP Integration	Context Management & Reliability

EXAM FOCUS

Built around real-world, production scenarios — testing practical architecture decisions over memorized theory.

REGISTER ON

Skilljar



FPT PARTNER ACCESS

Sign in with your FPT email on the Claude Partner Network.

HOW TO REGISTER

Three steps to your exam

1

Create your account

Register on Skilljar with your FPT account · anthropic.skilljar.com

2

Purchase the exam

Open the CCA-F exam page and buy your seat.

3

Get exam access

After payment, pick an available schedule from the booking flow to lock in your exam.

Choose a slot that suits you and sit the exam online or offline at a test center.

Exam Day & Proctoring Flow



BEFORE YOU START

Set up your space



- **Quiet, private space**
A meeting room or closed room works best.
- **Good lighting, clear desk**
Face well-lit, no extra materials around you.
- **Laptop fully charged**
Connect the charger before you begin.
- **Proctored & recorded**
A live Pearson VUE proctor monitors you in real time; the session is also recorded.

PROCTORING FLOW

Six steps on exam day

- 1 Download OnVUE & run the system test
- 2 Check in ~30 minutes early
- 3 Take your photo
- 4 Photograph your government ID
- 5 Complete the 360° room scan
- 6 Proctor launches your exam

The interface closely matches the official practice exam.

Results & Appeal Policy



RESULTS & TIMELINE

Your exam is scored automatically — here is what to expect.



1,000

SCALED SCORE

Scored on a 100–1,000 scale; 720 is the minimum to pass.



4

RETAKE LIMIT

Up to 4 attempts within 12 months. Wait 14 days after fail 1, 30 days after fail 2, and 90 days after fail 3.



12

MONTHS VALID

CCA-F stays valid for 12 months from the date you earn it.

03

New CCA-F Certification Update

Effective June 30, 2026

New CCA-F Certification Update



WHAT CHANGED

Effective June 30, 2026

Area	Previous Program	Current Program
Exam Fee	\$99	\$125 (+26%)
Retake Waiting Period	No retakes allowed	0 → 14 → 30 → 90 days
Certification Validity	6 Months	12 Months
Exam Platform	Skilljar + ProctorFree	Pearson VUE
Exam Delivery	Online Only	Online or Test Center
Previous Failures	6-Month Lockout	Attempt history reset

What didn't change

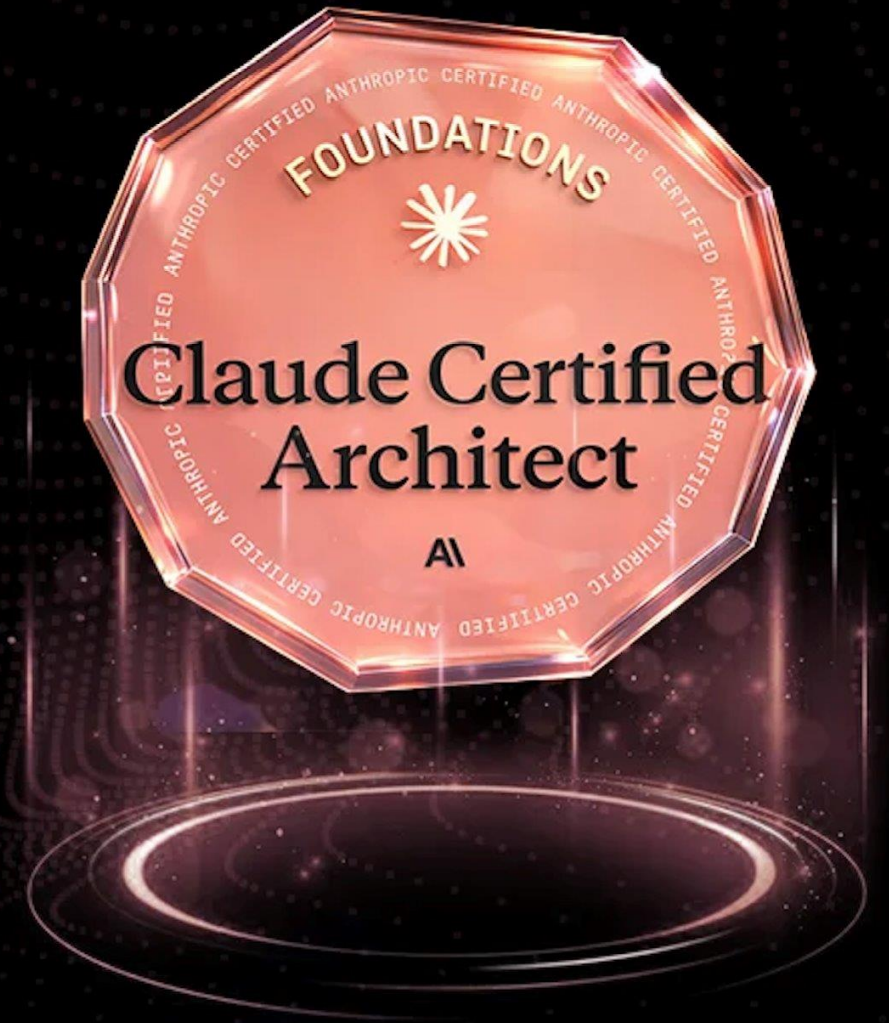
- 5 domains, same weightings
- Exam format & pass bar unchanged

04

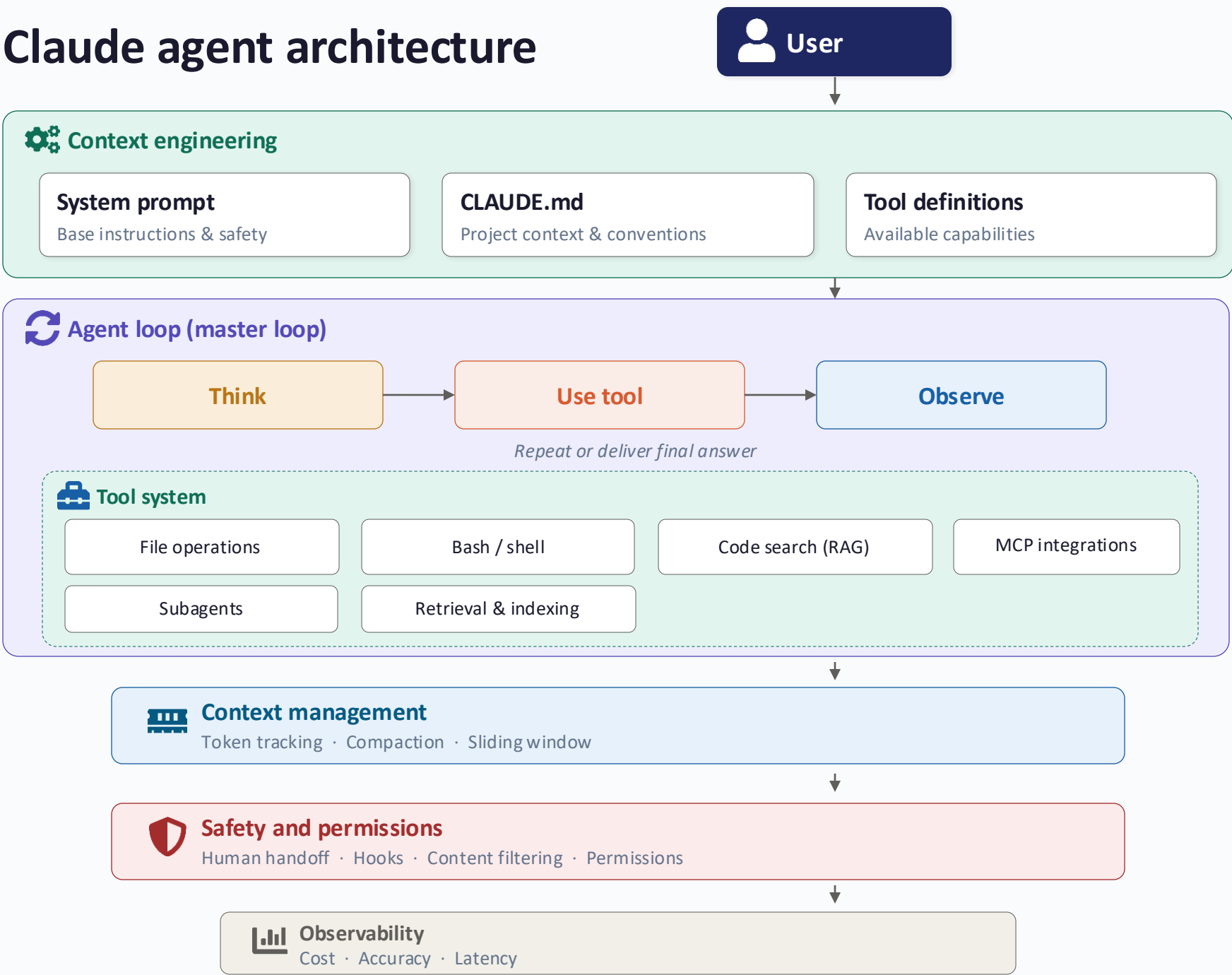
Tips

Claude Certified

**Architect – Foundations
(CCA-F)**



Claude agent architecture



Agent loop cycle

- Question
- Think
- Use tool
- Observe
- Think again
- Use another tool
- **Final answer**

Study What?

Domain 1: Agentic Architecture & Orchestration

Weight: 27% of scored content

Học gì?

1. Agentic loop: tiếp tục khi stop_reason == "tool_use", dừng khi stop_reason == "end_turn".
2. Coordinator / subagent orchestration: coordinator điều phối, subagent scope rõ, task độc lập chạy song song.
3. Human handoff: escalate khi user yêu cầu human, policy exception, hoặc agent không tiến triển.
4. Long-running workflow: có max_turns, progress check, fallback.
5. Multi-agent research: research → synthesis → gap detection → targeted re-research → final report.

Link Anthropic nên học

1. Building with the Claude API: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>
2. Claude Code in Action: <https://anthropic.skilljar.com/claude-code-in-action>
3. Introduction to MCP: <https://anthropic.skilljar.com/introduction-to-model-context-protocol>

Domain 2: Tool Design & MCP Integration

Weight: 18% of scored content

Anthropic, PBC · Confidential Need to Know (NTK)

Học gì?

1. Tool interface design: tool name/action rõ, description nêu khi nào dùng/không dùng.
2. Tool boundaries: tránh quá nhiều tool trùng nghĩa; consolidate hoặc split đúng lúc.
3. Error handling: structured error với errorCategory, isRetryable / retryable.
4. Tool choice: tool_choice="auto" có thể không gọi tool; dùng any hoặc force tool khi bắt buộc.
5. MCP: kết nối Claude với external tools/resources/prompts qua MCP server.

Link Anthropic nên học

1. Introduction to Model Context Protocol: <https://anthropic.skilljar.com/introduction-to-model-context-protocol>
2. Building with the Claude API: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>
3. Claude Code in Action: <https://anthropic.skilljar.com/claude-code-in-action>

Study What?



Domain 3: Claude Code Configuration & Workflows

Weight: 20% of scored content

Học gì?

1. Claude Code CLI: --max-turns, budget controls, permissions.
2. Project context: CLAUDE.md cho conventions; REVIEW.md cho review rules.
3. PR review architecture: static prompt để miss cross-file bugs; agentic review dùng tools tốt hơn.
4. Review quality: reduce false positives bằng conventions, examples, criteria.
5. Cost: Batch API cho bulk/offline reviews; split large PRs để tránh truncation.

Link Anthropic nên học:

1. Claude Code in Action: <https://anthropic.skilljar.com/claude-code-in-action>
2. Claude Code 101: <https://anthropic.skilljar.com/claude-code-101>
3. Building with the Claude API: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>

Domain 4: Prompt Engineering & Structured Output

Weight: 20% of scored content

Học gì?

1. Few-shot prompting: teach format, extraction behavior, and domain judgment.
2. Structured output: tool schema over "please output JSON".
3. Extraction quality: optional/nullable fields for absent source info.
4. Recovery: validation error feedback loop.
5. Prompt criteria: define what to report/skip; avoid over-filtering.

Link Anthropic nên học:

1. Building with the Claude API: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>
2. Claude 101: <https://anthropic.skilljar.com/claude-101>
3. AI Fluency: Framework & Foundations: <https://anthropic.skilljar.com/ai-fluency-framework-foundations>

Study What?



Domain 5: Context Management & Reliability

Weight: 15% of scored content

Học gì?

1. Context window: don't pass everything; use summaries, source index, reference IDs.
2. Freshness: stale tool results need re-fetch.
3. Reliability: aggregate accuracy can hide segment-level errors.
4. Cost/latency: batch vs real-time routing.
5. State: persist structured state, not just conversation logs.

Link Anthropic nên học

1. Building with the Claude API: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>
2. Claude Code in Action: <https://anthropic.skilljar.com/claude-code-in-action>
3. Introduction to MCP: <https://anthropic.skilljar.com/introduction-to-model-context-protocol>

6 Critical Concepts

1 When Code Beats a Prompt

- Prompts = guidance, Code = law
 - Prompts for style & formatting
 - Code, hooks & gates for hard rules
 - Critical workflows never rely on prompts alone
- ✗ “Enhance the system prompt” trap
- > wrong when money, compliance, or a hard business rule is involved.

2 Subagents Are Independent

- Each subagent starts with empty context
- They don't see your request or history
- Pass goal, inputs, results & format
- Never assume shared context

3 Tool Descriptions Drive Routing

- Claude routes by description, not name
 - Define purpose, I/O, when to use, edge cases
- Tool and MCP:
- Tool = actions
 - MCP Resource = read-only information

4 Where Information Gets Lost

- The “Lost in the Middle” effect
- Ends get attention; the middle is missed
- Buried instructions get overlooked
- Keep key facts on top; use summaries

5 Which API, and When

- Real-Time → user is waiting
 - Prompt Caching → repeated context, ~90% cheaper
 - Message Batches → background, 50% off, ≤24h
- Repeated + instant → Caching, not Batch

6 Spot the Wrong Answer First

- ✗ “Enhance the system prompt” for rules
- ✗ Batch API while a user is waiting
- ✗ A single agent holding every tool
- ✗ Let Claude self-report its confidence
- ✗ Run Claude Code in CI/CD with no flags
- ✗ Return a generic “Operation failed”
- ✓ **Code-enforced rules & structured errors**

Rules



Pattern	Chọn
Tool loop	Check stop_reason
Hết turn / không tiến triển	Escalate hoặc orchestration fallback
Customer yêu cầu human	Escalate ngay
Task độc lập	Parallel execution
Coordinator đã có context	Coordinator xử lý task đơn giản
Synthesis thiếu coverage	Coordinator detect gaps rồi re-delegate targeted search
Returning customer	Structured summary + fresh tool calls
Model bịa tool input	Update tool description / schema guardrail
Business error bị retry	Structured error retryable: false
Technical transient error	Retry/backoff ở tool/server layer
Nhiều tool gây confusion	Consolidate overlapping tools hoặc split by role
Generic tool free-text	Add enum parameter hoặc split purpose-specific tools
Need strict JSON	Define tool with target schema
Error text unstructured	Add structured metadata
Project convention false positives	CLAUDE.md
Skip generated files / CI checks	REVIEW.md
Cross-file bugs	Agentic review with read/search tools
Same session missed own bug	Same-session anchoring
Large PR output truncation	Split into multiple calls then merge
Need cap turns/spend	--max-turns + --max-budget-usd
Offline non-blocking PR review	Message Batches API

Rules



Pattern	Chọn
JSON compliance	Tool with schema
Field sometimes absent	Optional/nullable/null
Inconsistent extraction format	Few-shot examples
Validation error with fixable output	Follow-up with error
Model fabricates required field	Make field optional
Many false positives	Few-shot examples or explicit criteria
Missing bug due to severity filter	Add metadata + downstream filtering
Too much raw context	Filter/compress/summarize
Source attribution needed	Structured source index
Stale state	New session + summary + fresh tools
Aggregate accuracy	Analyze by segment/field
Offline cost reduction	Batch API
Urgent latency	Real-time Messages API
Resume after crash	Structured export + manifest

Mental model to do the Exam

Keyword	Domain	Cách chọn đáp án
stop_reason, loop, tool_use, end_turn, max_turns	Agentic Architecture	Dựa vào agent loop chuẩn
subagent, coordinator, synthesis, parallel tasks	Agentic Orchestration	Coordinator điều phối; task độc lập thì chạy song song
tool schema, tool description, tool_choice, error metadata	Tool Design	Tool rõ boundary, schema strict, error có retryable
MCP, external system, server, resources, prompts	MCP Integration	MCP kết nối Claude với tools/resources/prompts bên ngoài
Claude Code, GitHub Action, PR review, CLAUDE.md, REVIEW.md	Claude Code	Persistent project context + review config
JSON, schema, extraction, validation	Structured Output	Tool use với input schema đáng tin hơn prompt "return JSON"
hallucination, missing fields, optional/required	Structured Output / Reliability	Field có thể thiếu thì optional/null
long context, stale tool results, memory	Context Management	Compress/summarize/filter/re-fetch fresh data
batch, cost, latency	Cost Optimization	Offline/bulk → Message Batches API; urgent → Messages API

Scenario & Tips



Scenario	Tip trả lời
Extraction nhiều doc types, <code>tool_choice=auto</code> trả text	Dùng <code>tool_choice=any</code> để buộc gọi tool
Enum <code>property_type</code> thiếu type mới	Add <code>other</code> + detail field
Calendar invite cần JSON strict	Define tool with target schema
Resume extraction strict schema	Extract from <code>tool_use</code> , không regex text
Retry validation error nào ít hiệu quả	Không thể recover info không có trong input
4 subagents đều access 18 tools	Too many tools hurt selection
Generic document tool không reliable	Add <code>analysis_type</code> enum
Subagent definitions có nhưng không execute	Missing <code>Task</code> in allowed tools
Agent bịa <code>order_id</code> khi refund	Tool description: <code>order_id</code> from prior <code>lookup_order</code> , never assumed
Tool error "operation failed"	Add <code>errorCategory</code> , <code>isRetryable</code> , explanation
Business errors bị retry vô ích	Return <code>retriable: false</code> + customer-friendly message
MCP tools tăng, accuracy drop vì overlap	Consolidate semantically overlapping tools

Scenario & Tips



Scenario	Tip trả lời
Static prompt review misses cross-file bugs	Redesign as turn-limited agentic task with read/search tools
Review flags intentional project patterns	Put accepted patterns in CLAUDE.md
50 PR/day, review non-blocking, cost high	Batch ok if feedback up to 24h later still actionable
Same Claude session misses bugs later caught by CI	Same-session anchoring / self-consistency bias
Review misses branch-level coverage gaps	Enumerate branches/exception paths and verify tests
Noisy review: generated code, CI checks, rendering patterns	Create REVIEW.md with skip rules and evidence requirements
Test generation creates low-value tests	Document testing standards in CLAUDE.md
One prompt covers security/API/business logic; trade-off recall	Split focused prompts by concern then consolidate
False positives in code review	Few-shot examples distinguish acceptable vs genuine issues
200 commit summaries, 12h latency acceptable	Message Batches API with custom_ids
Need analyze dismissed findings	Add detected_pattern metadata
Critical-only prompt suppresses real bugs	Report findings with confidence/severity; filter downstream
Many findings dismissed as not worth addressing	Define explicit report vs skip criteria
Huge PR reviews run too long/costly	--max-turns 10 --max-budget-usd 2.00
Large PR output truncated mid-JSON	Split review into multiple calls and merge findings

Scenario & Tips



Scenario	Tip trả lời
Follow-up summary chậm vì spawn synthesis subagent và pass 80K tokens	Coordinator đã có context thì tự summarize, không spawn subagent không cần thiết
Generic analyze_document dùng free-text instruction gây output không ổn định	Thêm analysis_type enum hoặc mode rõ ràng
Legal case có nhiều precedents, xử lý sequential quá chậm	Chạy parallel subagents, mỗi agent xử lý subset, rồi aggregate
Multi-agent research crash giữa chừng, cần resume	Persist structured export + manifest; coordinator inject relevant state
Synthesis subagent cần findings từ agent khác	Pass reference IDs + shared memory store
Coordinator nói sẽ delegate nhưng subagent không chạy	Kiểm tra allowedTools có Task
Web search và document analysis độc lập nhưng chạy tuần tự	Emit cả hai Task tool calls trong cùng response
Simple fact query đi qua full pipeline	Coordinator dynamically decide subagents cần invoke
Synthesis phát hiện research gaps	Coordinator re-delegate targeted search rồi synthesize lại
Agent quên thông tin xác thực	API stateless; phải pass conversation history
Billing dispute cần human escalation	Structured handoff với customer/order/issue
User muốn human nhưng issue có thể xử lý ngay	Acknowledge + offer resolve now or escalate
Implement agent loop	Loop dựa trên stop_reason
Returning customer, tool results stale	New session + structured summary + fresh tool calls
Escalation logic	Escalate khi user asks human / policy exception / no progress
User nói muốn người thật ngay	Immediate escalate_to_human
Backend refund lỗi nhưng có đủ info	Explain, confirm eligibility, acknowledge system issue, offer retry/escalation
Agent hết max_turns chưa resolve	Orchestration-layer outcome check và auto escalate
Human không có transcript	Pass structured summary, không pass full transcript

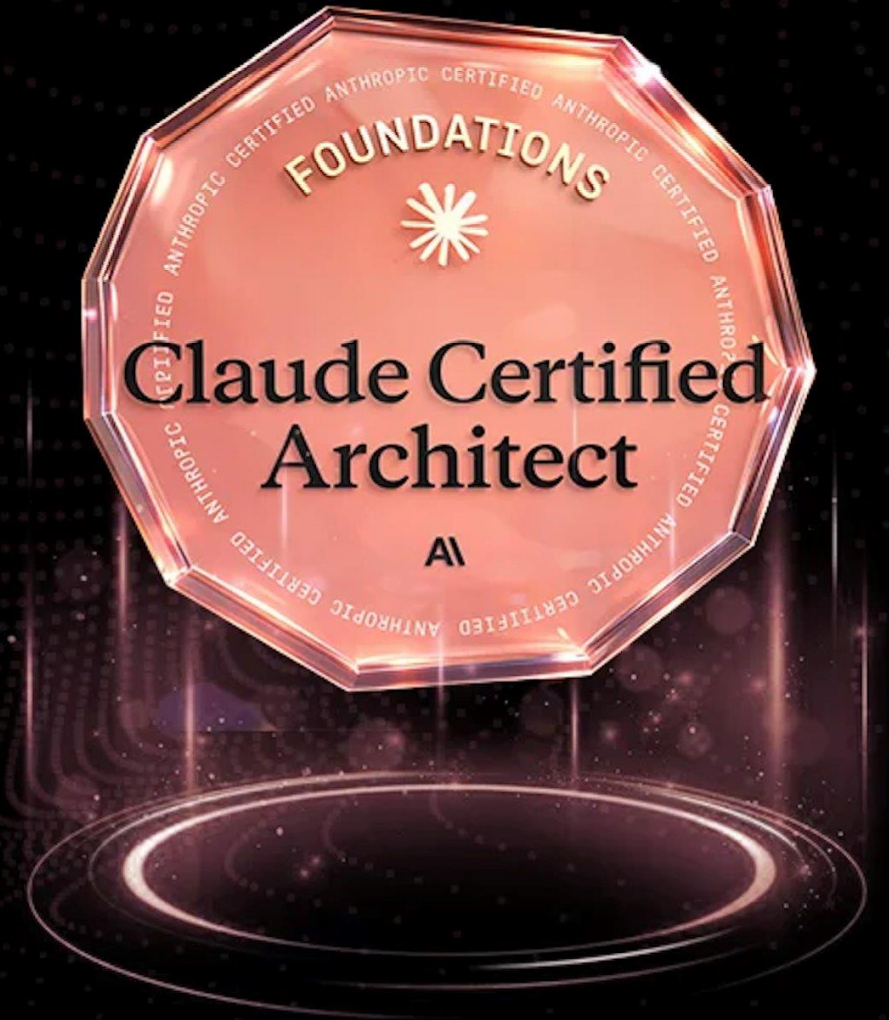
Scenario & Tips



Scenario	Tip trả lời
Standard monthly reports + urgent exception reports	Standard → Batch; urgent → real-time
97% aggregate high-confidence accuracy	Validate by document type and field
Source attribution lost	Preserve source metadata
Summary slow due to 80K context transfer	Coordinator handles simple summary directly
Resume after crash	Structured exports + manifest
Passing findings to synthesis	Reference IDs + shared memory
Final report needs citations with token budget	Pass synthesis draft + source index
Returning customer with stale tool result	New session + summary + fresh calls
Many large order tool responses	Extract only relevant fields
Human handoff after long conversation	Structured summary

05

Real Scenarios in Exam Claude Certified Architect – Foundations (CCA-F)



When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 17 precedents takes over 3 minutes. What is the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- A. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity.
- B. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents.
- C. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations.
- D. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis.**

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization (e.g., the report generator tries to analyze documents). What is the primary cause of this poor tool selection behavior?

- A. Choosing from 18 tools instead of 4–5 relevant ones increases decision complexity beyond reliable selection thresholds.**
- B. The agents' role descriptions in their system prompts conflict with having access to tools outside that role.
- C. The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.
- D. The tool definitions consume too much context window space, leaving insufficient room for task content.

The automated review consistently flags patterns your team uses intentionally — force-unwrapping optionals in test files, using large coordinator classes that follow your maintained modules marked as deprecated in the public SDK. Developers are dismissing roughly 30% of all findings as project-specific false positives. Which approach most effectively addresses these false positives in the first place by supplying the project's conventions as persistent context on every review?

A. Document the team's accepted patterns and intentional conventions in the project's CLAUDE.md file so the model receives this context during every review.

B. Build post-processing keyword filters that suppress findings containing terms like "force unwrap," "large class," or "deprecated import" before results reach developers.

C. Configure the review to analyze only the changed lines in the diff without surrounding file context, reducing the amount of code the model evaluates per review.

D. Have developers add inline suppress-comments at flagged lines and preprocess diffs to exclude suppressed lines before sending code to the model.

The extraction pipeline receives documents of varying types—some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with a schema tailored to a specific document type. During testing, you observe that with `tool_choice: "auto"`, Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need structured output without knowing the document type in advance. What's the most effective approach?

- A. Set `tool_choice: "any"` with all extraction tools defined.
- B. Keep `tool_choice: "auto"` with system prompt instructions requiring tool use.
- C. Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.**
- D. Consolidate all document types into a single unified-schema extraction tool and force that tool.

Your agent has called `lookup_order` multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history, etc.) and now occupies a majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

- A. Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues.
- B. Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details.**
- C. Proceed with additional lookups without modifying the existing tool output context.
- D. Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions.

References



1. [claude-architect-foundations-en](#)
2. [NotebookLM \(push Exam Guide\)](#)
3. [Anthropic Certification Exam Policy](#)
4. [Claude Certified Architect – Foundations – Exam Guide](#)
5. [Certification Terms and Conditions](#)

06

Q&A

Questions, answers, and discussion